

TRANSACT-SQL (T-SQL) COMPREHENSIVE LANGUAGE REFERENCE

Complete Syntax, Data Manipulation, Stored Procedures, and Optimization Standard

SECTION 1.0 T-SQL LANGUAGE OVERVIEW AND DATA TYPES

Transact-SQL (T-SQL) is Microsoft's proprietary extension for SQL Server.

Standard Data Types:

- Exact Numerics: ``BIGINT`` (8 bytes), ``INT`` (4 bytes), ``SMALLINT`` (2 bytes), ``DECIMAL(p,s)`` / ``NUMERIC(p,s)``, ``MONEY``.
- Character Strings: ``VARCHAR(n)`` (non-Unicode dynamic), ``NVARCHAR(n)`` (Unicode UTF-16, 2 bytes per char), ``VARCHAR(MAX)`` (up to 2GB).
- Date and Time: ``DATETIME2`` (100ns precision, recommended over legacy ``DATETIME``), ``DATE``, ``TIME``, ``DATETIMEOFFSET``.
- Binary and Special: ``VARBINARY(MAX)``, ``UNIQUEIDENTIFIER`` (GUID), ``XML``, ``JSON`` (stored as `NVARCHAR` with ``ISJSON()`` validation).

SECTION 2.0 DATA QUERY LANGUAGE (DQL) AND ADVANCED JOINS

The fundamental query syntax follows the logical execution order: FROM -> ON -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> TOP/OFFSET.

Join Types:

- ``INNER JOIN``: Returns rows where join condition matches in both tables.
- ``LEFT OUTER JOIN``: Returns all rows from left table and matched rows from right (or NULL).
- ``FULL OUTER JOIN``: Returns rows when there is a match in either table.
- ``CROSS APPLY`` and ``OUTER APPLY``: Invokes a table-valued function or subquery for each outer row (similar to correlated subquery with multiple column return).

SECTION 3.0 COMMON TABLE EXPRESSIONS (CTE) AND WINDOW FUNCTIONS

Common Table Expressions (CTEs) simplify complex modular logic and recursive hierarchies.

Syntax: ``WITH CTE_Name AS (SELECT Column1, Column2 FROM TableName) SELECT * FROM CTE_Name``;

Recursive CTEs consist of an Anchor member, ``UNION ALL``, and a Recursive member referencing the CTE name.

Window Functions compute aggregations over specified partitions without collapsing rows:

- `ROW_NUMBER() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC)`: Assigns unique sequential integers.
- `RANK()` and `DENSE_RANK()`: Handle ties with gaps and without gaps respectively.
- `LEAD(col, 1) OVER (...)` and `LAG(col, 1) OVER (...)`: Access following or preceding row values.

SECTION 4.0 STORED PROCEDURES, FUNCTIONS, AND TRIGGERS

Stored Procedures provide precompiled execution plans, modularity, and security separation.

Example Stored Procedure with parameters and output:

```
`CREATE PROCEDURE dbo.usp_ProcessOrder @OrderId INT, @Status NVARCHAR(50) OUTPUT AS BEGIN
SET NOCOUNT ON; UPDATE Orders SET Status = @Status WHERE OrderId = @OrderId; SET @Status =
'SUCCESS'; END;`
```

User-Defined Functions (UDF):

- Scalar UDFs: Return single value. Avoid in large WHERE clauses due to row-by-row invocation penalties.
- Inline Table-Valued Functions (iTVF): Return table variable directly; inlined by optimizer like a view.

DML Triggers: `AFTER INSERT, UPDATE, DELETE` or `INSTEAD OF` triggers operating on virtual `inserted` and `deleted` tables.

SECTION 5.0 TRANSACTIONS, CONCURRENCY, AND ERROR HANDLING

Transactions enforce ACID guarantees (Atomicity, Consistency, Isolation, Durability).

Transaction Syntax with Structured Exception Handling:

```
`BEGIN TRY BEGIN TRANSACTION; -- Statements COMMIT TRANSACTION; END TRY BEGIN CATCH IF
@@TRANCOUNT > 0 ROLLBACK TRANSACTION; THROW; END CATCH;`
```

Transaction Isolation Levels:

- `READ UNCOMMITTED` (Dirty Reads allowed).
- `READ COMMITTED` (Default; prevents dirty reads using shared read locks).
- `READ COMMITTED SNAPSHOT (RCSI)`: Row-versioning in tempdb; non-blocking reads.
- `REPEATABLE READ`: Shared locks held until end of transaction.
- `SERIALIZABLE`: Key-range locking; highest isolation, lowest concurrency.

SECTION 6.0 INDEXING ARCHITECTURE AND T-SQL OPTIMIZATION

Indexing is critical for fast retrieval and reducing I/O bottlenecks:

- Clustered Index: Sorts and stores data rows in the table based on their key values (one per table).
- Non-Clustered Index: Separate structure containing index keys and row locators (pointers to heap or clustered key).
- Covering Index (`INCLUDE` clause): Adds non-key columns to leaf level to eliminate Key Lookups.

Best Practices for High-Performance T-SQL:

1. Ensure predicates are SARGable (Search Argument Able): Avoid `WHERE YEAR(CreateDate) = 2026`; use `WHERE CreateDate >= '2026-01-01' AND CreateDate < '2027-01-01'`.

2. Avoid `SELECT \*`; explicitly list required column names to minimize memory grants and network traffic.

3. Use `SET NOCOUNT ON` in stored procedures to suppress row count messages.